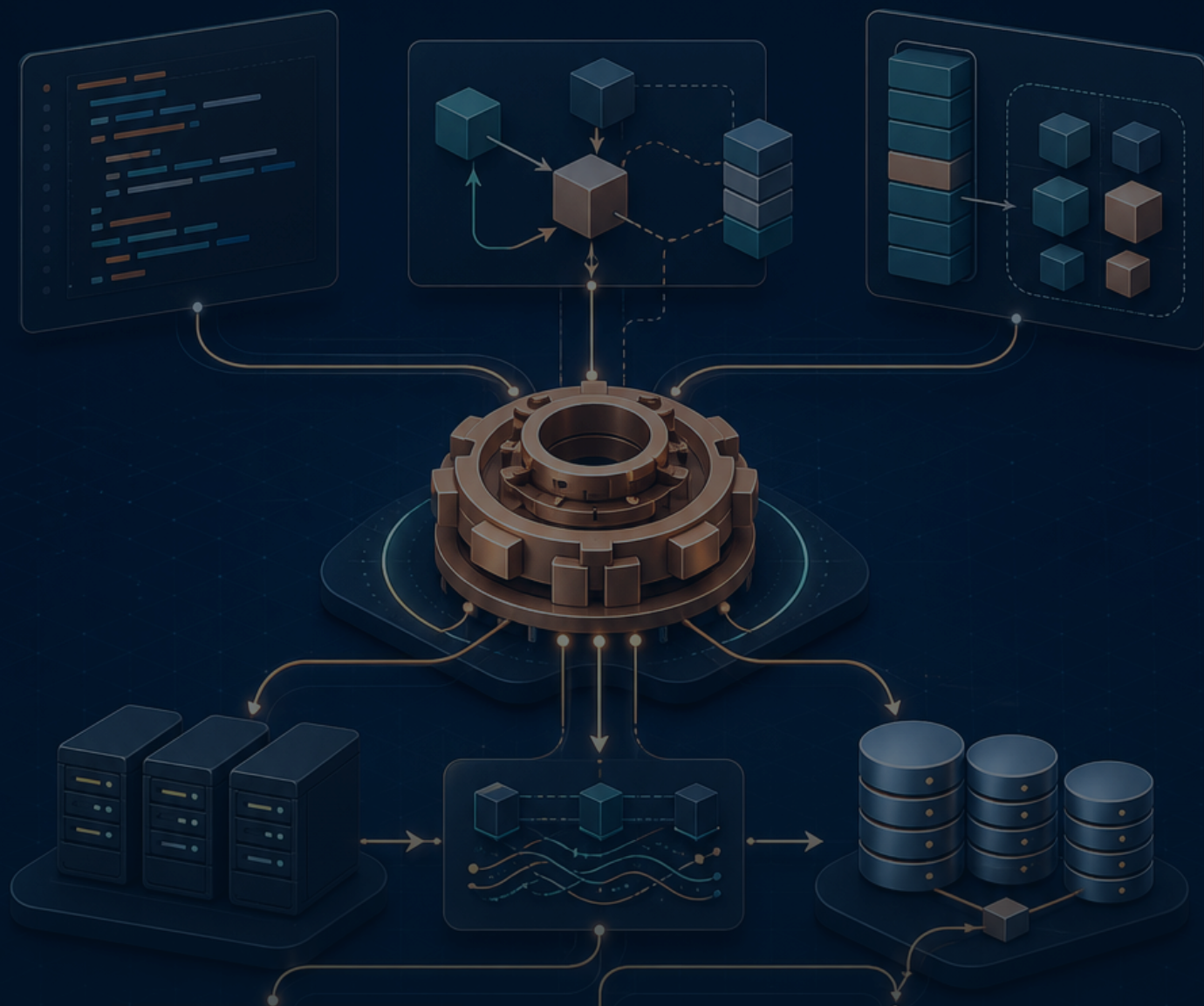




VOLUME 09 / RUST BACKEND MASTERY

RustNote — 要件設計・構成・セットアップ

要件、画面、domain、schema、module、依存、起動、route、UI、healthを実装前に一つの設計へ揃える

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

動く雛形だけで終わらず、完成判断、失敗状態、依存方向、アクセシビリティ、診断経路を先に作ります。

到達目標

- ✓ 受入条件をtestへ変換できる
- ✓ domain/schema/UIの不変条件を揃えられる
- ✓ 依存方向と起動順を説明できる
- ✓ 最初の縦切りをsmoke testできる

学習地図

01 要件・user story・受入条件

RustNoteで誰が何を達成するかをuser storyへし、正常系だけでなく未認証、空入力、他人ID、DB障害を受入条件に含めます。画面より先に行動と不変条件を決めます。

02 画面遷移・情報設計・wireframe

register、login、一覧、作成、編集をURLと遷移で整理します。各画面に主目的の一つ置き、空状態、loading、error、成功、mobile幅を設計します。

03 domain model・schema・不変条件

User、Note、Sessionをdomain型とDB tableへ対応させます。email一意、title長、owner、session期限、cascade削除を型・アプリ検証・DB制約の三層で一致させます。

04 project構造と依存方向

srcをweb、application、domain、infrastructureへ分けます。Topcoatとlibsqlは外側に置き、domainがframeworkやDBを知らない依存方向にします。初心者向けには過分割せず責務で切ります。

05 Cargo.toml・version固定・feature

TopcoatとCLIは0.5.0へ固定し、libsql、argon2、serde、tracing、uuid等の責務を依存表で説明します。依存を追加した理由とfeatureを記録し、cargo treeで重複を確認します。

06 Config・AppState・DB初期化

mainではConfigを一度検証し、DB接続とmigrationを行い、共有handleをAppStateへ入れてrouterを作ります。handler内で環境変数やBuilderを作り直しません。

07 migration・seed・test database

migrationは本番とtestの同じSQLを使い、seedは開発専用の明示コマンドにします。testごとに隔離DBを作り、順序依存をなくします。

08 route inventoryと実装順

routeをmethod、path、認証、入力、成功、失敗で一覧化し、health→layout→auth→list→create→edit→toggle→deleteの薄い縦切りで実装します。

09 design system・assets・accessibility

色、space、font、focus、button、form、card、errorを少数tokenとcomponentへまとめます。semantic HTML、label、error association、keyboard focus、contrastを最初から確認します。

10 health・first run・診断

livenessはprocessが応答可能か、readinessは必要依存を使えるかを分けま。公開healthへ秘密や詳細errorを出さず、内部logで段階を追います。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

personaと目的

STEP 2

story

STEP 3

acceptance criteria

STEP 4

非機能要件

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

要件・user story・受入条件

UNIT 01・CONCEPT

要件・user story・受入条件

RustNoteで誰が何を達成するかをuser storyへし、正常系だけでなく未認証、空入力、他人ID、DB障害を受入条件に含めます。画面より先に行動と不変条件を決めます。

この単元の到達点

- personaと目的
- story
- acceptance criteria
- 非機能要件

なぜバックエンドで必要か

受入条件をintegration test名へ変換し、route、SQL、画面の完成判断を一貫させます。



UNIT 01・MENTAL MODEL

要件・user story・受入条件を図でつかむ



RustNoteで誰が何を達成するかをuser storyへし、正常系だけでなく未認証、空入力、他人ID、DB障害を受入条件に含めます。画面より先に行動と不変条件を決めます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	personaと目的	personaと目的 — 入力と前提を確認し、境界を曖昧にしない。
2	story	story — 値がどの順で変換されるかを追跡する。
3	acceptance criteria	acceptance criteria — 失敗を再現し、型またはログから原因を特定する。
4	非機能要件	非機能要件 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

要件・user story・受入条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

要件・user story・受入条件だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
Story: 利用者として自分のnoteを作成したい
Given login済み
When titleを入力しPOST /notes
Then 303で一覧へ戻り自分のnoteだけ表示
And 空titleは400、他人のnoteは404
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
Story: 利用者として自分のnoteを作成したい	この行が受け取る値、作る値、副作用の有無を確認します。
Given login済み	この行が受け取る値、作る値、副作用の有無を確認します。
When titleを入力しPOST /notes	この行が受け取る値、作る値、副作用の有無を確認します。
Then 303で一覧へ戻り自分のnoteだけ表示	この行が受け取る値、作る値、副作用の有無を確認します。
And 空titleは400、他人のnoteは404	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力personaと目的の条件を満たすか確認する。
- ✓ storyを実行し、途中の型と状態を追う。
- ✓ acceptance criteriaで失敗を成功経路から分離する。
- ✓ 非機能要件により、出力と副作用を検証する。

型を声に出す 要件・user story・受入条件に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

実装前に正常・境界・拒否・障害の期待が決まります。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
edit_own_note_returns_303
edit_empty_title_returns_400_with_value
edit_other_users_note_returns_404
edit_missing_note_returns_404
edit_db_failure_returns_503_and_logs_request_id
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01 · BACKEND APPLICATION

バックエンドのどこで使うか

受入条件をintegration test名へ変換し、route、SQL、画面の完成判断を一貫させます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	personaと目的に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	storyの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	acceptance criteriaまたは非機能要件の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

要件・user story・受入条件を再現して切り分ける

要件・user story・受入条件の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: personaと目的
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: story
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: acceptance criteria
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 非機能要件

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	personaと目的	personaと目的 — 入力と前提を確認し、境界を曖昧にしない。
2	story	story — 値がどの順で変換されるかを追跡する。
3	acceptance criteria	acceptance criteria — 失敗を再現し、型またはログから原因を特定する。
4	非機能要件	非機能要件 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 実装前に利用者の成果と拒否条件を観測可能な受入条件へします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 ・ PRACTICE

手を動かす演習

編集機能のuser storyと、正常・空・他人・存在なし・DB失敗の受入条件を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
edit_own_note_returns_303
edit_empty_title_returns_400_with_value
edit_other_users_note_returns_404
edit_missing_note_returns_404
edit_db_failure_returns_503_and_logs_request_id
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 実装前に利用者の成果と拒否条件を観測可能な受入条件へします。
- ✓ 要件・user story・受入条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

画面遷移・情報設計・wireframe

UNIT 02 · CONCEPT

画面遷移・情報設計・wireframe

register、login、一覧、作成、編集をURLと遷移で整理します。各画面に主目的を一つ置き、空状態、loading、error、成功、mobile幅を設計します。

この単元の到達点

- URL map
- 主行動
- 状態別UI
- keyboardとmobile

なぜバックエンドで必要か

Topcoat SSRではserverが完成HTMLを返します。validation error時は同じformへ値とerrorを再描画し、focusをerror summaryへ移します。



UNIT 02 · MENTAL MODEL

画面遷移・情報設計・wireframeを図でつかむ



register、login、一覧、作成、編集をURLと遷移で整理します。各画面に主目的を一つ置き、空状態、loading、error、成功、mobile幅を設計します。図では左から右へ処理を追います。各箱の入口では入力のと信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	URL map	URL map — 入力と前提を確認し、境界を曖昧にしない。
2	主行動	主行動 — 値がどの順で変換されるかを追跡する。
3	状態別UI	状態別UI — 失敗を再現し、型またはログから原因を特定する。
4	keyboardとmobile	keyboardとmobile — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

画面遷移・情報設計・wireframeとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE まず動く最小コード

画面遷移・情報設計・wireframeだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
GET /register -> POST /register -> 303 /login
GET /login -> POST /login -> 303 /notes
GET /notes -> POST /notes -> 303 /notes
GET /notes/{id}/edit -> POST /notes/{id}/edit -> 303 /notes
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
GET /register -> POST /register -> 303 /login	この行が受け取る値、作る値、副作用の有無を確認します。
GET /login -> POST /login -> 303 /notes	この行が受け取る値、作る値、副作用の有無を確認します。
GET /notes -> POST /notes -> 303 /notes	この行が受け取る値、作る値、副作用の有無を確認します。
GET /notes/{id}/edit -> POST /notes/{id}/edit -> 303 /notes	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がURL mapの条件を満たすか確認する。
- ✓ 主行動を実行し、途中の型と状態を追う。
- ✓ 状態別UIで失敗を成功経路から分離する。
- ✓ keyboardとmobileにより、出力と副作用を検証する。

型を声に出す 画面遷移・情報設計・wireframeに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

画面遷移・情報設計・wireframe

デスクトップとmobileのwireframeで、見出し・form・note card・error summaryの位置が確認できます。

観察ポイント

- URL map
- 主行動
- 状態別UI

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
register: h1アカウント作成/submit/項目error/login link
login: h1ログイン/submit/generic error/register link
notes: h1ノート/作成form/空状態/edit・toggle・delete
edit: h1編集/save/cancel
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02・BACKEND APPLICATION

バックエンドのどこで使うか

Topcoat SSRではserverが完成HTMLを返します。validation error時は同じformへ値とerrorを再描画し、focusをerror summaryへ移します。



リクエスト側

- URL mapを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 主行動を小さな責務へ分割
- 状態別UIをResultとログで表現
- keyboardとmobileをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	URL mapに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	主行動の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	状態別UIまたはkeyboardとmobileの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

画面遷移・情報設計・wireframeを再現して切り分ける

画面遷移・情報設計・wireframeの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: URL map
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 主行動
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 状態別UI
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: keyboardとmobile

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	URL map	URL map — 入力と前提を確認し、境界を曖昧にしない。
2	主行動	主行動 — 値がどの順で変換されるかを追跡する。
3	状態別UI	状態別UI — 失敗を再現し、型またはログから原因を特定する。
4	keyboardとmobile	keyboardとmobile — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 URLと画面状態を先に設計し、各ページの主目的と失敗表示を固定します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

全画面の見出し、主ボタン、空状態、error位置、次の遷移を表にしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
register: h1アカウント作成/submit/項目error/login link  
login: h1ログイン/submit/generic error/register link  
notes: h1ノート/作成form/空状態/edit・toggle・delete  
edit: h1編集/save/cancel
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ URLと画面状態を先に設計し、各ページの主目的と失敗表示を固定します。
- ✓ 画面遷移・情報設計・wireframeとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

domain model・schema・不変条件

UNIT 03・CONCEPT

domain model・schema・不変条件

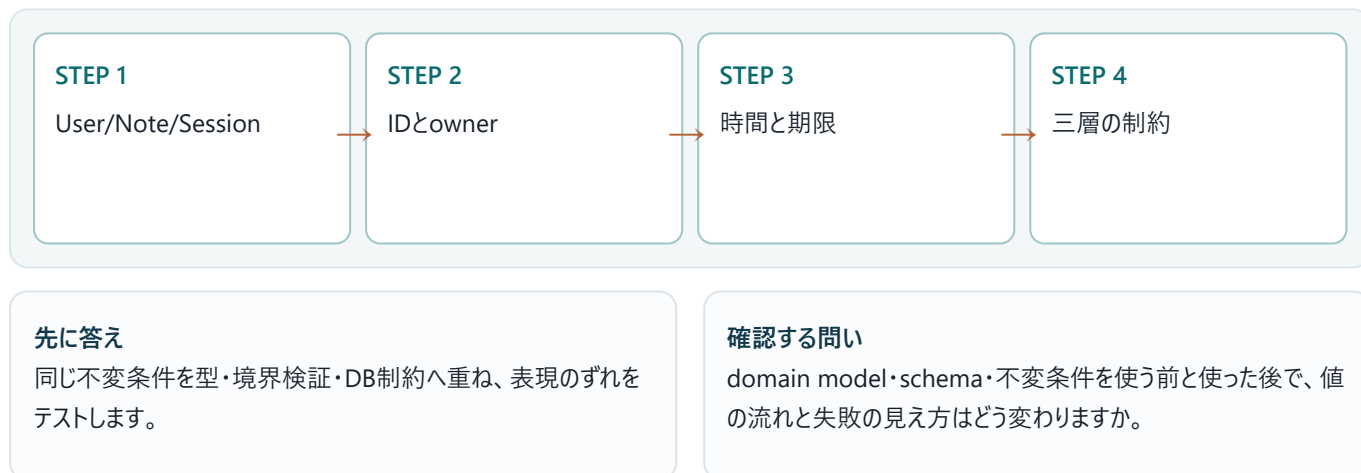
User、Note、Sessionをdomain型とDB tableへ対応させます。email一意、title長、owner、session期限、cascade削除を型・アプリ検証・DB制約の三層で一致させます。

この単元の到達点

- User/Note/Session
- IDとowner
- 時間と期限
- 三層の制約

なぜバックエンドで必要か

web DTOとDB rowをdomainから分け、password_hashとtoken_hashが画面用型へ入らないようにします。



UNIT 03・MENTAL MODEL

domain model・schema・不変条件を図でつかむ



User、Note、Sessionをdomain型とDB tableへ対応させます。email一意、title長、owner、session期限、cascade削除を型・アプリ検証・DB制約の三層で一致させます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	User/Note/Session	User/Note/Session — 入力と前提を確認し、境界を曖昧にしない。
2	IDとowner	IDとowner — 値がどの順で変換されるかを追跡する。
3	時間と期限	時間と期限 — 失敗を再現し、型またはログから原因を特定する。
4	三層の制約	三層の制約 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

domain model・schema・不変条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03・MINIMUM CODE

まず動く最小コード

domain model・schema・不変条件だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
users(id,email,password_hash,created_at)
notes(id,user_id,title,body,done,created_at,updated_at)
sessions(token_hash,user_id,expires_at,created_at)
UNIQUE users(email)
CHECK note title 1..80
FK notes/sessions user_id ON DELETE CASCADE
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>users(id,email,password_hash,created_at)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>notes(id,user_id,title,body,done,created_at,updated_at)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>sessions(token_hash,user_id,expires_at,created_at)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>UNIQUE users(email)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>CHECK note title 1..80</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力>User/Note/Sessionの条件を満たすか確認する。
- ✓ IDとownerを実行し、途中の型と状態を追う。
- ✓ 時間と期限で失敗を成功経路から分離する。
- ✓ 三層の制約により、出力と副作用を検証する。

型を声に出す domain model・schema・不変条件に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

ER図: User 1 — * Note / User 1 — * Session

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
Rust: NoteTitle::parse 1..80 chars
Form: 項目errorを400で表示
SQL: CHECK(length(trim(title)) BETWEEN 1 AND 80)
Test: empty/81 chars/日本語80 chars
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

web DTOとDB rowをdomainから分け、password_hashとtoken_hashが画面用型へ入らないようにします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	User/Note/Sessionに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	IDとownerの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	時間と期限または三層の制約の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

domain model・schema・不変条件を再現して切り分ける

domain model・schema・不変条件の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: User/Note/Session
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: IDとowner
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 時間と期限
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 三層の制約

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	User/Note/Session	User/Note/Session — 入力と前提を確認し、境界を曖昧にしない。
2	IDとowner	IDとowner — 値がどの順で変換されるかを追跡する。
3	時間と期限	時間と期限 — 失敗を再現し、型またはログから原因を特定する。
4	三層の制約	三層の制約 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 同じ不変条件を型・境界検証・DB制約へ重ね、表現のずれをテストします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

Noteの不変条件をRust生成関数、form validation、SQL CHECKの三つで表してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
Rust: NoteTitle::parse 1..80 chars  
Form: 項目errorを400で表示  
SQL: CHECK(length(trim(title)) BETWEEN 1 AND 80)  
Test: empty/81 chars/日本語80 chars
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 同じ不変条件を型・境界検証・DB制約へ重ね、表現のずれをテストします。
- ✓ domain model・schema・不変条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

project構造と依存方向

UNIT 04 · CONCEPT

project構造と依存方向

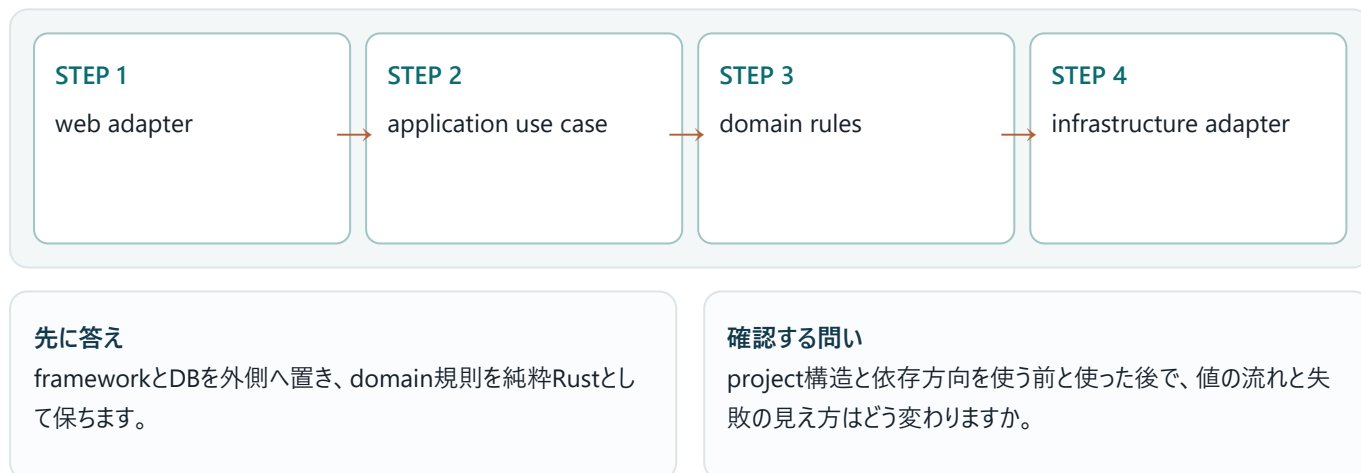
srcをweb、application、domain、infrastructureへ分けます。Topcoatとlibsqlは外側に置き、domainがframeworkやDBを知らない依存方向にします。初心者向けには過分割せず責務で切ります。

この単元の到達点

- web adapter
- application use case
- domain rules
- infrastructure adapter

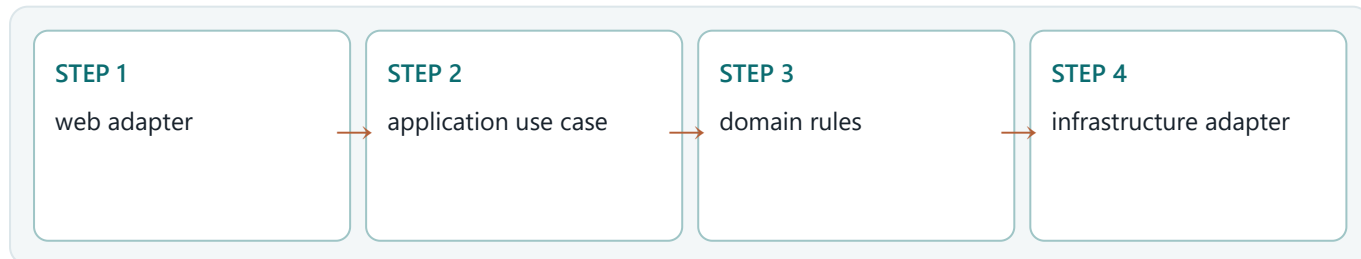
なぜバックエンドで必要か

小規模なうちは一package内moduleにし、独立deployやcompile boundaryが必要になってからworkspace分割します。



UNIT 04 · MENTAL MODEL

project構造と依存方向を図でつかむ



srcをweb、application、domain、infrastructureへ分けます。Topcoatとlibsqlは外側に置き、domainがframeworkやDBを知らない依存方向にします。初心者向けには過分割せず責務で切ります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	web adapter	web adapter — 入力と前提を確認し、境界を曖昧にしない。
2	application use case	application use case — 値がどの順で変換されるかを追跡する。
3	domain rules	domain rules — 失敗を再現し、型またはログから原因を特定する。
4	infrastructure adapter	infrastructure adapter — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

project構造と依存方向とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

project構造と依存方向だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
src/
main.rs      # config/router/start
web/         # Topcoat pages/routes/views
application/ # create/update/list use cases
domain/      # User/Note/value objects/errors
infrastructure/ # libsql repository/argon2
app.rs       # AppState helpers
assets/app.css
migrations/
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
src/	この行が受け取る値、作る値、副作用の有無を確認します。
main.rs # config/router/start	この行が受け取る値、作る値、副作用の有無を確認します。
web/ # Topcoat pages/routes/views	この行が受け取る値、作る値、副作用の有無を確認します。
application/ # create/update/list use cases	この行が受け取る値、作る値、副作用の有無を確認します。
domain/ # User/Note/value objects/errors	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力web adapterの条件を満たすか確認する。
- ✓ application use caseを実行し、途中の型と状態を追う。
- ✓ domain rulesで失敗を成功経路から分離する。
- ✓ infrastructure adapterにより、出力と副作用を検証する。

型を声に出す project構造と依存方向に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

変更理由ごとの置き場所が決まり、routeからSQLが直接見えません。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
web::create(Form<Input>) -> application::create_note(actor, command, &repo)
application::create_note -> NoteTitle::parse -> repo.insert(owner,title)
repo returns Note -> web maps to SeeOther or AppError response
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

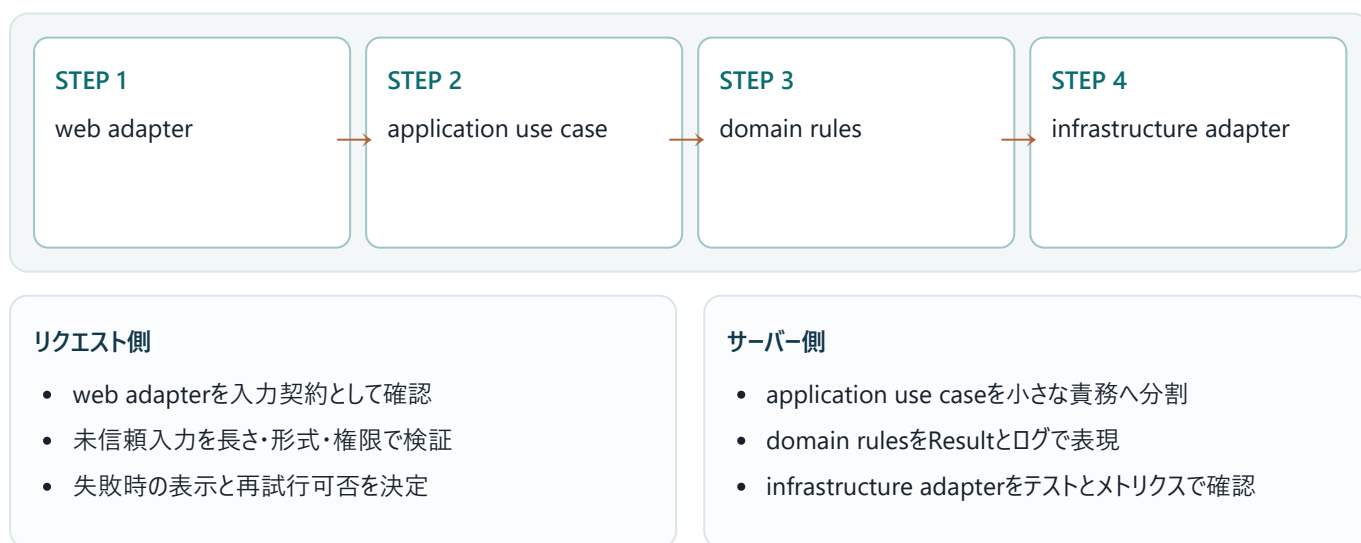
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

小規模なうちは一package内moduleにし、独立deployやcompile boundaryが必要になってからworkspace分割します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	web adapterに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	application use caseの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	domain rulesまたはinfrastructure adapterの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

project構造と依存方向を再現して切り分ける

project構造と依存方向の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: web adapter
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: application use case
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: domain rules
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: infrastructure adapter

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	web adapter	web adapter — 入力と前提を確認し、境界を曖昧にしない。
2	application use case	application use case — 値がどの順で変換されるかを追跡する。
3	domain rules	domain rules — 失敗を再現し、型またはログから原因を特定する。
4	infrastructure adapter	infrastructure adapter — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 frameworkとDBを外側へ置き、domain規則を純粋Rustとして保ちます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 ・ PRACTICE

手を動かす演習

create_noteの入力がweb→application→domain→repositoryへ流れるuse宣言と関数署名を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
web::create(Form<Input>) -> application::create_note(actor, command, &repo)
application::create_note -> NoteTitle::parse -> repo.insert(owner, title)
repo returns Note -> web maps to SeeOther or AppError response
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ frameworkとDBを外側へ置き、domain規則を純粋Rustとして保ちます。
- ✓ project構造と依存方向とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

Cargo.toml・version固定・feature

UNIT 05 ・ CONCEPT

Cargo.toml・version固定・feature

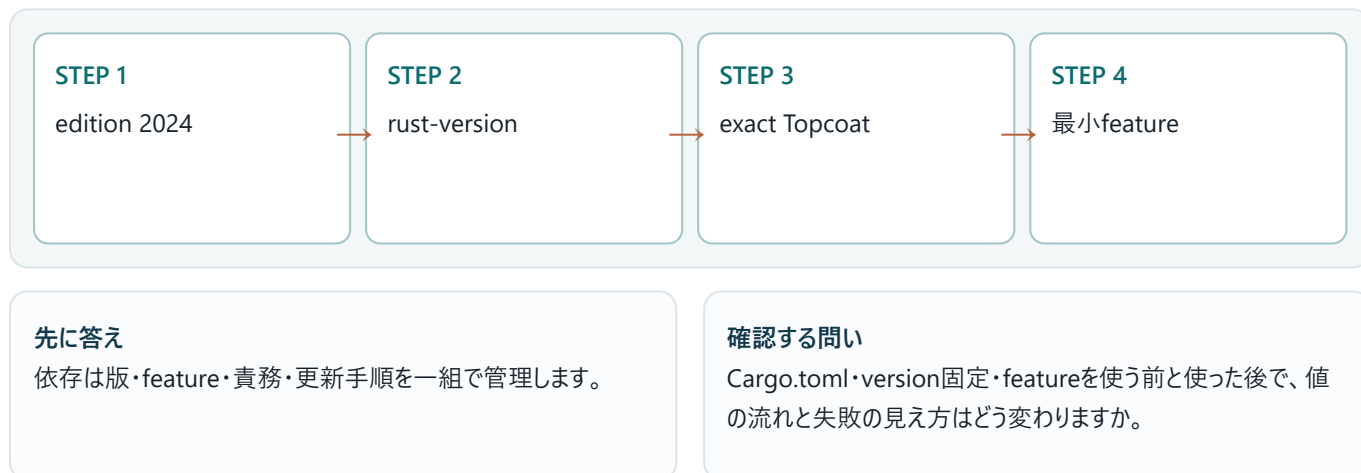
TopcoatとCLIは0.5.0へ固定し、libsql、argon2、serde、tracing、uuid等の責務を依存表で説明します。依存を追加した理由とfeatureを記録し、cargo treeで重複を確認します。

この単元の到達点

- edition 2024
- rust-version
- exact Topcoat
- 最小feature

なぜバックエンドで必要か

Cargo.lockをcommitし、Dependabot等の更新は一件ずつtest・smoke・migration互換を確認します。



UNIT 05 ・ MENTAL MODEL

Cargo.toml・version固定・featureを図でつかむ



TopcoatとCLIは0.5.0へ固定し、libsql、argon2、serde、tracing、uuid等の責務を依存表で説明します。依存を追加した理由とfeatureを記録し、cargo treeで重複を確認します。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	edition 2024	edition 2024 — 入力と前提を確認し、境界を曖昧にしない。
2	rust-version	rust-version — 値がどの順で変換されるかを追跡する。
3	exact Topcoat	exact Topcoat — 失敗を再現し、型またはログから原因を特定する。
4	最小feature	最小feature — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Cargo.toml・version固定・featureとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE
まず動く最小コード

Cargo.toml・version固定・featureだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
[package]
edition="2024"
rust-version="1.95"
[dependencies]
topcoat="0.5.0"
tokio={version="1",features=["rt-multi-thread","macros","signal"]}
libsql="0.9.29"
argon2="0.5.3"
serde={version="1",features=["derive"]}
tracing="0.1"
```

●●● 実行コマンド

```
cargo tree -d
cargo check --locked --all-targets
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
[package]	この行が受け取る値、作る値、副作用の有無を確認します。
edition="2024"	この行が受け取る値、作る値、副作用の有無を確認します。
rust-version="1.95"	この行が受け取る値、作る値、副作用の有無を確認します。
[dependencies]	この行が受け取る値、作る値、副作用の有無を確認します。
topcoat="0.5.0"	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がedition 2024の条件を満たすか確認する。
- ✓ rust-versionを実行し、途中の型と状態を追う。
- ✓ exact Topcoatで失敗を成功経路から分離する。
- ✓ 最小featureにより、出力と副作用を検証する。

型を声に出す Cargo.toml・version固定・featureに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

```
cargo check --locked --all-targets → success
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
topcoat: HTTP/SSR/session/assets
libsql: Turso接続/SQL
argon2: password hash
serde: form/JSON変換
tracing: 構造化log
依存追加前にstd/既存crateで代替できないか確認
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

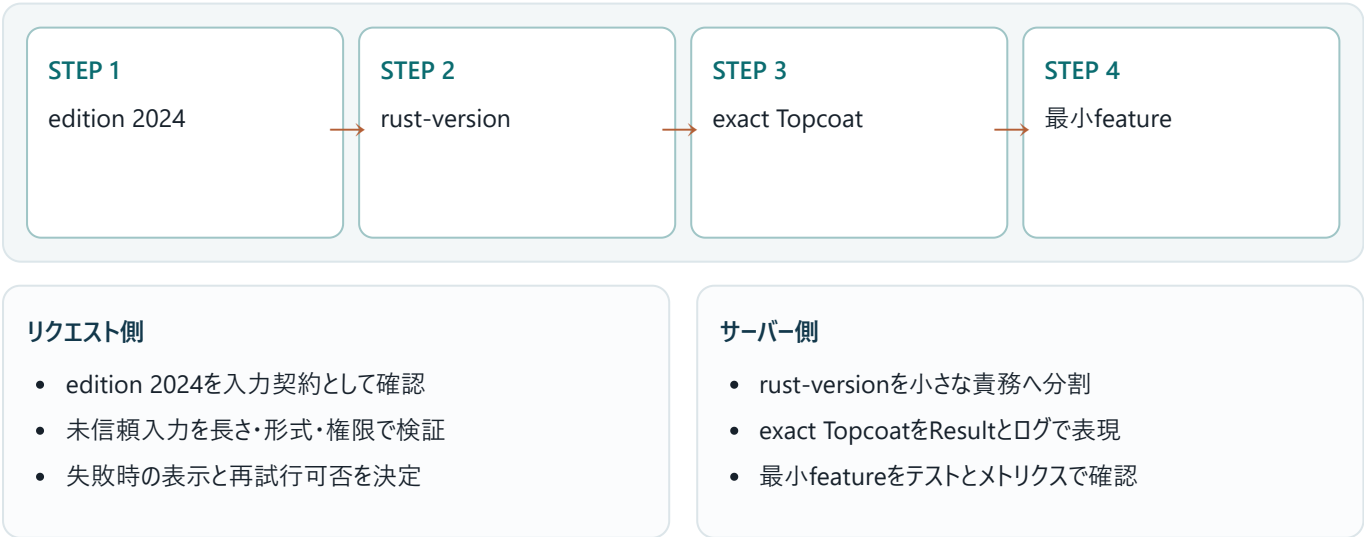
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05・BACKEND APPLICATION

バックエンドのどこで使うか

Cargo.lockをcommitし、Dependabot等の更新は一件ずつtest・smoke・migration互換を確認します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	edition 2024に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	rust-versionの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	exact Topcoatまたは最小featureの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

Cargo.toml・version固定・featureを再現して切り分ける

Cargo.toml・version固定・featureの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: edition 2024
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: rust-version
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: exact Topcoat
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 最小feature

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	edition 2024	edition 2024 — 入力と前提を確認し、境界を曖昧にしない。
2	rust-version	rust-version — 値がどの順で変換されるかを追跡する。
3	exact Topcoat	exact Topcoat — 失敗を再現し、型またはログから原因を特定する。
4	最小feature	最小feature — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 依存は版・feature・責務・更新手順を一組で管理します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

各依存の役割と、削除したらどの機能が失われるかを表にしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
topcoat: HTTP/SSR/session/assets
libsql: Turso接続/SQL
argon2: password hash
serde: form/JSON変換
tracing: 構造化log
依存追加前にstd/既存crateで代替できないか確認
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 依存は版・feature・責務・更新手順を一組で管理します。
- ✓ Cargo.toml・version固定・featureとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

Config・AppState・DB初期化

UNIT 06・CONCEPT

Config・AppState・DB初期化

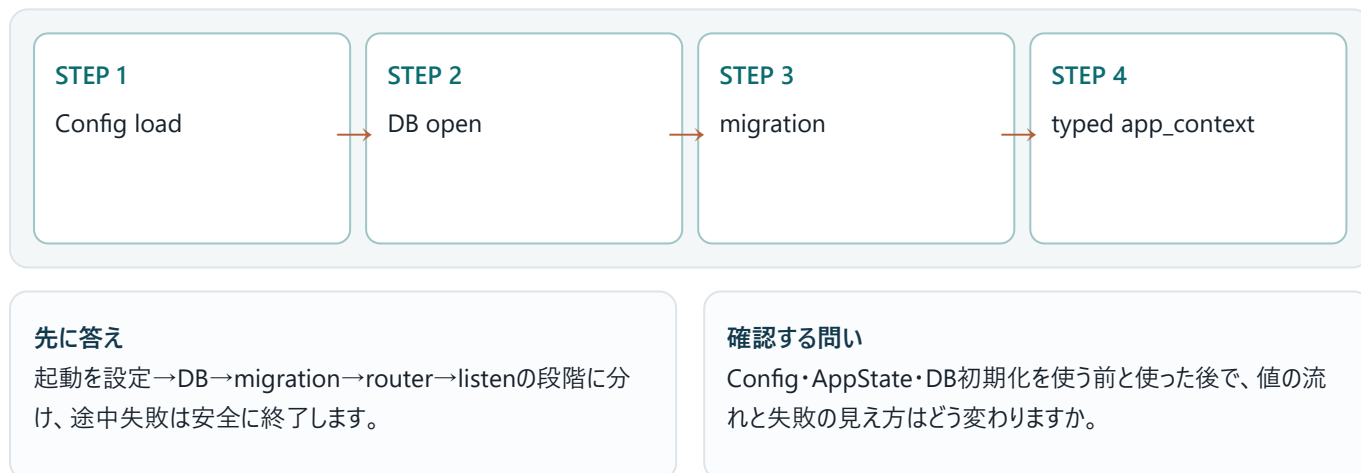
mainではConfigを一度検証し、DB接続とmigrationを行い、共有handleをAppStateへ入れてrouterを作ります。handler内で環境変数やBuilderを作り直しません。

この単元の到達点

- Config load
- DB open
- migration
- typed app_context

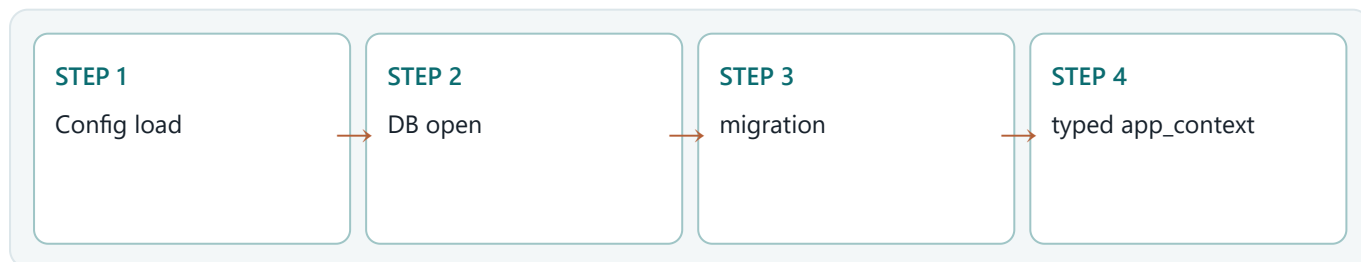
なぜバックエンドで必要か

起動途中で失敗したらportを開かず終了し、どの段階で失敗したか構造化logへ残します。秘密値は表示しません。



UNIT 06・MENTAL MODEL

Config・AppState・DB初期化を図でつかむ



mainではConfigを一度検証し、DB接続とmigrationを行い、共有handleをAppStateへ入れてrouterを作ります。handler内で環境変数やBuilderを作り直しません。図では左から右へ処理を追います。各箱の入口では入力の方と信頼度、出口では結果の方と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Config load	Config load — 入力と前提を確認し、境界を曖昧にしない。
2	DB open	DB open — 値がどの順で変換されるかを追跡する。
3	migration	migration — 失敗を再現し、型またはログから原因を特定する。
4	typed app_context	typed app_context — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

Config・AppState・DB初期化とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE まず動く最小コード

Config・AppState・DB初期化だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::main]async fn main()->Result<(),Box<dyn std::error::Error>>{init_tracing();let config=Config::load()?;let db=open_database(&config).await?.migrate(&db).await?.let state=AppState::new(config,db);let router=build_router(state)?.topcoat::start(router).await?.Ok({})}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::main]async fn main()->Result<(),Box<dyn std::error::Error>>{init_</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
Config・AppState・DB初期化	入力、処理、出力を分離して読みます。
Config・AppState・DB初期化	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がConfig loadの条件を満たすか確認する。
- ✓ DB openを実行し、途中の型と状態を追う。
- ✓ migrationで失敗を成功経路から分離する。
- ✓ typed app_contextにより、出力と副作用を検証する。

型を声に出す Config・AppState・DB初期化に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
INFO config loaded host=0.0.0.0 port=3000
INFO migrations complete version=3
INFO server started
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
let config=Config::load().map_err(|e|StartupError::Config(e))?;let
db=open(&config).await.map_err(|e|StartupError::Database(e))?;migrate(&db).await.map_err(|e|StartupError::Mi
gration(e))?;
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

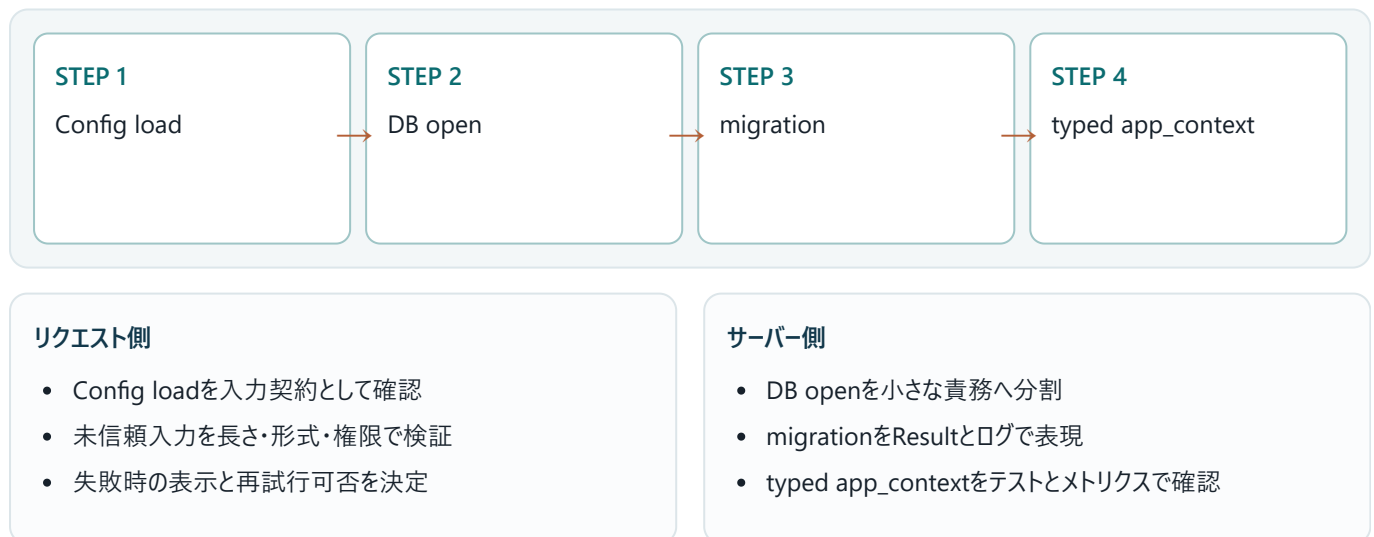
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

起動途中で失敗したらportを開かず終了し、どの段階で失敗したか構造化logへ残します。秘密値は表示しません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Config loadに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	DB openの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	migrationまたはtyped app_contextの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

Config・AppState・DB初期化を再現して切り分ける

Config・AppState・DB初期化の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Config load
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: DB open
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: migration
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: typed app_context

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Config load	Config load — 入力と前提を確認し、境界を曖昧にしない。
2	DB open	DB open — 値がどの順で変換されるかを追跡する。
3	migration	migration — 失敗を再現し、型またはログから原因を特定する。
4	typed app_context	typed app_context — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 起動を設定→DB→migration→router→listenの段階に分け、途中失敗は安全に終了します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

Config不正、DB接続失敗、migration失敗の三つを別error contextで返してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
let config=Config::load().map_err(|e|StartupError::Config(e))?;let
db=open(&config).await.map_err(|e|StartupError::Database(e))?;migrate(&db).await.map_err(|e|StartupError::Mi
gration(e))?;
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 起動を設定→DB→migration→router→listenの段階に分け、途中失敗は安全に終了します。
- ✓ Config・AppState・DB初期化とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

migration・seed・test database

UNIT 07 ・ CONCEPT

migration・seed・test database

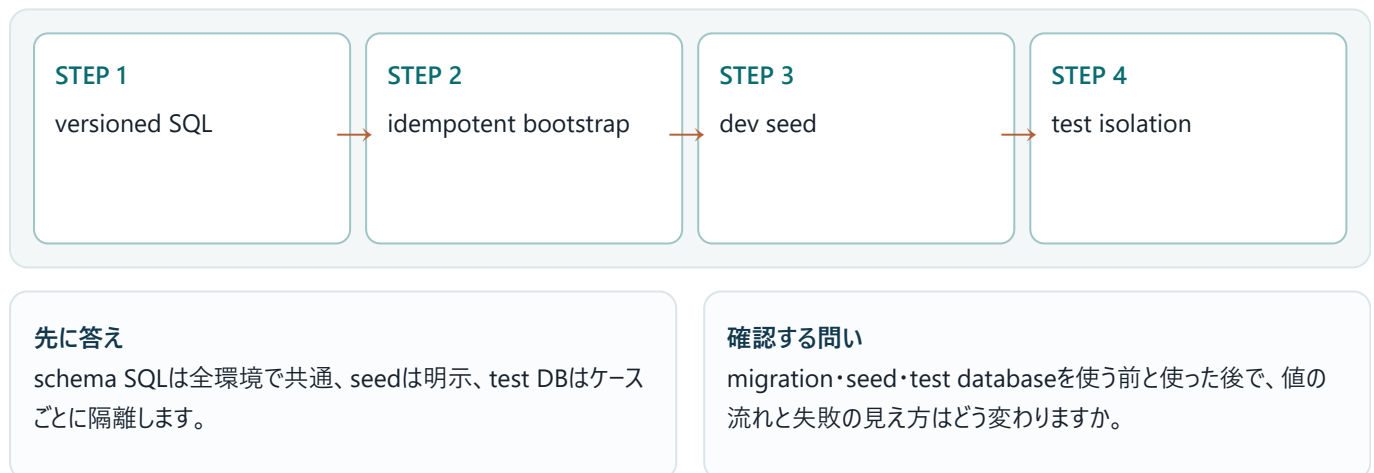
migrationは本番とtestの同じSQLを使い、seedは開発専用の明示コマンドにします。testごとに隔離DBを作り、順序依存をなくします。

この単元の到達点

- versioned SQL
- idempotent bootstrap
- dev seed
- test isolation

なぜバックエンドで必要か

本番起動時にsample userを自動作成せず、開発seedをfeatureまたは別binaryへ分けます。



UNIT 07 ・ MENTAL MODEL

migration・seed・test databaseを図でつかむ



migrationは本番とtestの同じSQLを使い、seedは開発専用の明示コマンドにします。testごとに隔離DBを作り、順序依存をなくします。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	versioned SQL	versioned SQL — 入力と前提を確認し、境界を曖昧にしない。
2	idempotent bootstrap	idempotent bootstrap — 値がどの順で変換されるかを追跡する。
3	dev seed	dev seed — 失敗を再現し、型またはログから原因を特定する。
4	test isolation	test isolation — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

migration・seed・test databaseとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE まず動く最小コード

migration・seed・test databaseだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
migrations/0001_users.sql
migrations/0002_notes.sql
migrations/0003_sessions.sql

async fn test_db()->Database{let
db=Builder::new_local(":memory:").build().await.unwrap();migrate(&db).await.unwrap();db}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
migrations/0001_users.sql	この行が受け取る値、作る値、副作用の有無を確認します。
migrations/0002_notes.sql	この行が受け取る値、作る値、副作用の有無を確認します。
migrations/0003_sessions.sql	この行が受け取る値、作る値、副作用の有無を確認します。
async fn test_db()->Database{let db=Builder::new_local(":memory:").build() }	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力versioned SQLの条件を満たすか確認する。
- ✓ idempotent bootstrapを実行し、途中の型と状態を追う。
- ✓ dev seedで失敗を成功経路から分離する。
- ✓ test isolationにより、出力と副作用を検証する。

型を声に出す migration・seed・test databaseに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

testごとに空schemaが作られ、seed user/noteだけが存在します。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
CREATE TABLE IF NOT EXISTS schema_migrations(version INTEGER PRIMARY KEY,applied_at TEXT NOT NULL);
-- transaction内でversion未適用を確認→SQL実行→version INSERT→COMMIT
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

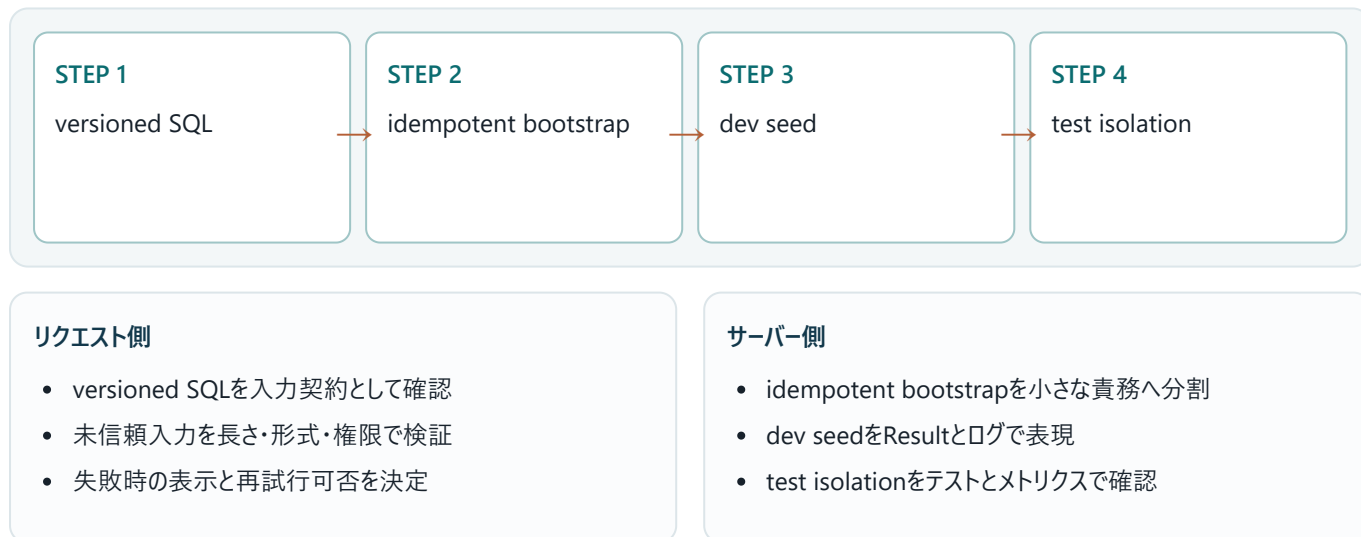
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

本番起動時にsample userを自動作成せず、開発seedをfeatureまたは別binaryへ分けます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	versioned SQLに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	idempotent bootstrapの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	dev seedまたはtest isolationの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

migration・seed・test databaseを再現して切り分ける

migration・seed・test databaseの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: versioned SQL
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: idempotent bootstrap
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: dev seed
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: test isolation

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	versioned SQL	versioned SQL — 入力と前提を確認し、境界を曖昧にしない。
2	idempotent bootstrap	idempotent bootstrap — 値がどの順で変換されるかを追跡する。
3	dev seed	dev seed — 失敗を再現し、型またはログから原因を特定する。
4	test isolation	test isolation — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 schema SQLは全環境で共通、seedは明示、test DBはケースごとに隔離します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

同じmigrationを二回適用しても履歴表により一度だけ実行されるrunnerを設計してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
CREATE TABLE IF NOT EXISTS schema_migrations(version INTEGER PRIMARY KEY, applied_at TEXT NOT NULL);  
-- transaction内でversion未適用を確認→SQL実行→version INSERT→COMMIT
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ schema SQLは全環境で共通、seedは明示、test DBはケースごとに隔離します。
- ✓ migration・seed・test databaseとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

route inventoryと実装順

UNIT 08 · CONCEPT

route inventoryと実装順

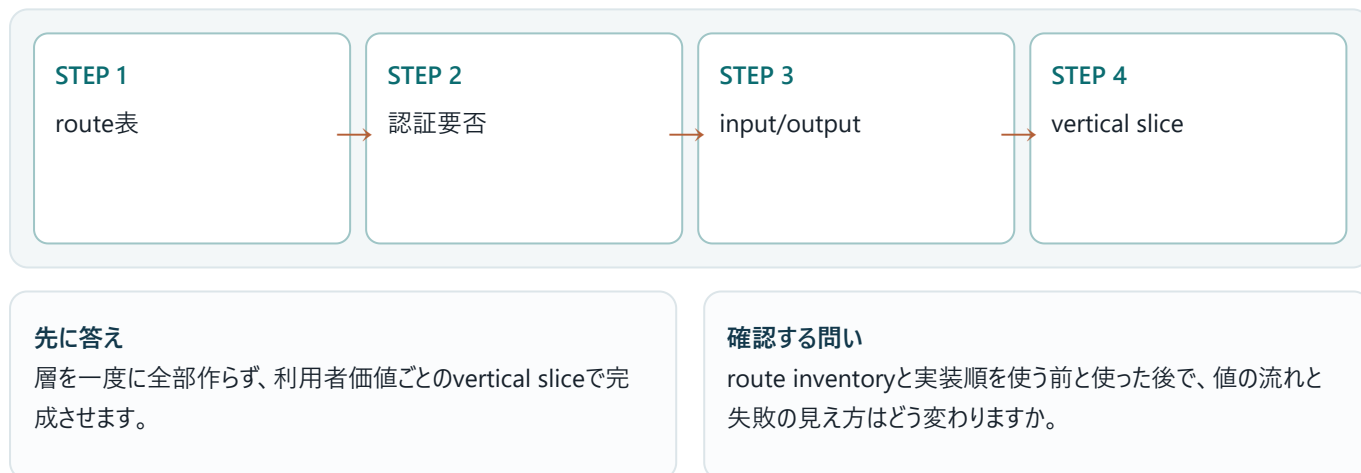
routeをmethod、path、認証、入力、成功、失敗で一覧化し、
health→layout→auth→list→create→edit→toggle→deleteの薄い縦切りで実装します。

この単元の到達点

- route表
- 認証可否
- input/output
- vertical slice

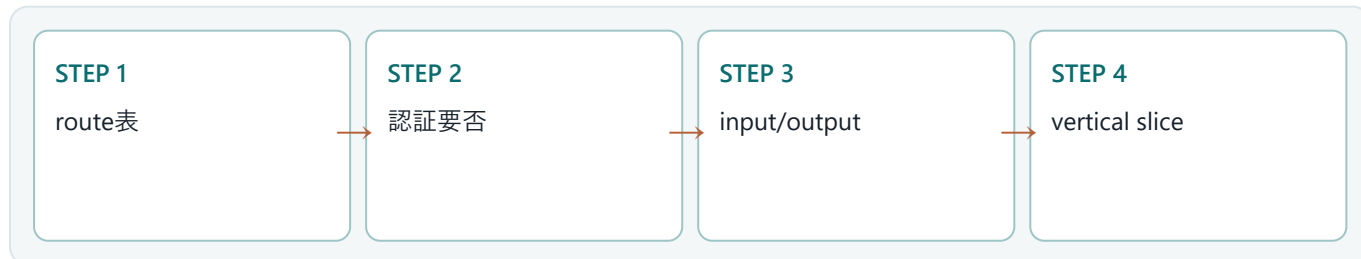
なぜバックエンドで必要か

route表はHTTP契約と実装順の両方を示し、各縦切りでweb・application・repository・view・testを同時に完成させます。



UNIT 08 · MENTAL MODEL

route inventoryと実装順を図でつかむ



routeをmethod、path、認証、入力、成功、失敗で一覧化し、health→layout→auth→list→create→edit→toggle→deleteの薄い縦切りで実装します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	route表	route表 — 入力と前提を確認し、境界を曖昧にしない。
2	認証可否	認証可否 — 値がどの順で変換されるかを追跡する。
3	input/output	input/output — 失敗を再現し、型またはログから原因を特定する。
4	vertical slice	vertical slice — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

route inventoryと実装順とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08 · MINIMUM CODE

まず動く最小コード

route inventoryと実装順だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
GET /health public -> 200 text
GET /login public -> HTML
POST /login public -> 303/400/429
GET /notes auth -> HTML
POST /notes auth -> 303/400
POST /notes/{note_id}/delete auth+owner -> 303/404
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
GET /health public -> 200 text	この行が受け取る値、作る値、副作用の有無を確認します。
GET /login public -> HTML	この行が受け取る値、作る値、副作用の有無を確認します。
POST /login public -> 303/400/429	この行が受け取る値、作る値、副作用の有無を確認します。
GET /notes auth -> HTML	この行が受け取る値、作る値、副作用の有無を確認します。
POST /notes auth -> 303/400	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 route 表の条件を満たすか確認する。
- ✓ 認証要否を実行し、途中の型と状態を追う。
- ✓ input/output で失敗を成功経路から分離する。
- ✓ vertical slice により、出力と副作用を検証する。

型を声に出す route inventory と実装順に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

一機能ごとに route、use case、SQL、view、test が同時に完成します。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 08 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
RouteSpec{method:"POST",path:"/notes/{note_id}/delete",auth:true,input:"NoteId path",success:"303 /notes",failures:["400 invalid id","302 login","404 not owned","503 db"]}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

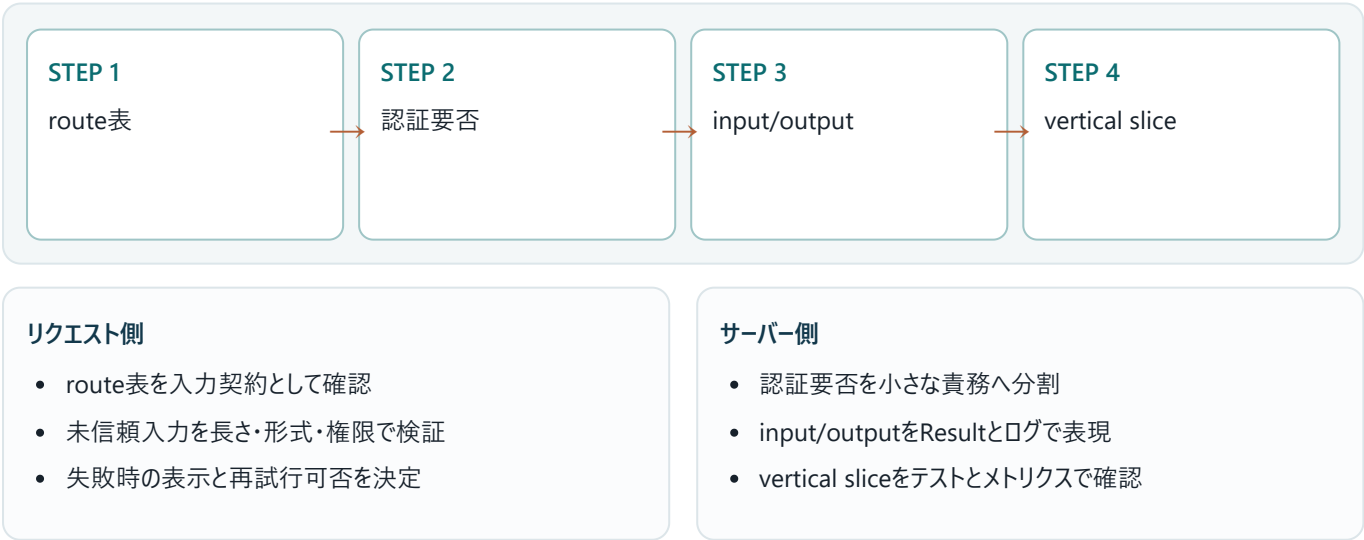
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

route表はHTTP契約と実装順の両方を示し、各縦切りでweb・application・repository・view・testを同時に完成させます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	route表に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	認証要否の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	input/outputまたはvertical sliceの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

route inventoryと実装順を再現して切り分ける

route inventoryと実装順の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: route表
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 認証要否
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: input/output
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: vertical slice

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	route表	route表 — 入力と前提を確認し、境界を曖昧にしない。
2	認証要否	認証要否 — 値がどの順で変換されるかを追跡する。
3	input/output	input/output — 失敗を再現し、型またはログから原因を特定する。
4	vertical slice	vertical slice — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 層を一度に全部作らず、利用者価値ごとのvertical sliceで完成させます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

全routeに404、400、401/redirect、403/404、503のどれが起こるかを記入します。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
RouteSpec{method:"POST",path:"/notes/{note_id}/delete",auth:true,input:"NoteId path",success:"303 /notes",failures:["400 invalid id","302 login","404 not owned","503 db"]}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 層を一度に全部作らず、利用者価値ごとのvertical sliceで完成させます。
- ✓ route inventoryと実装順とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

design system・assets・accessibility

UNIT 09・CONCEPT

design system・assets・accessibility

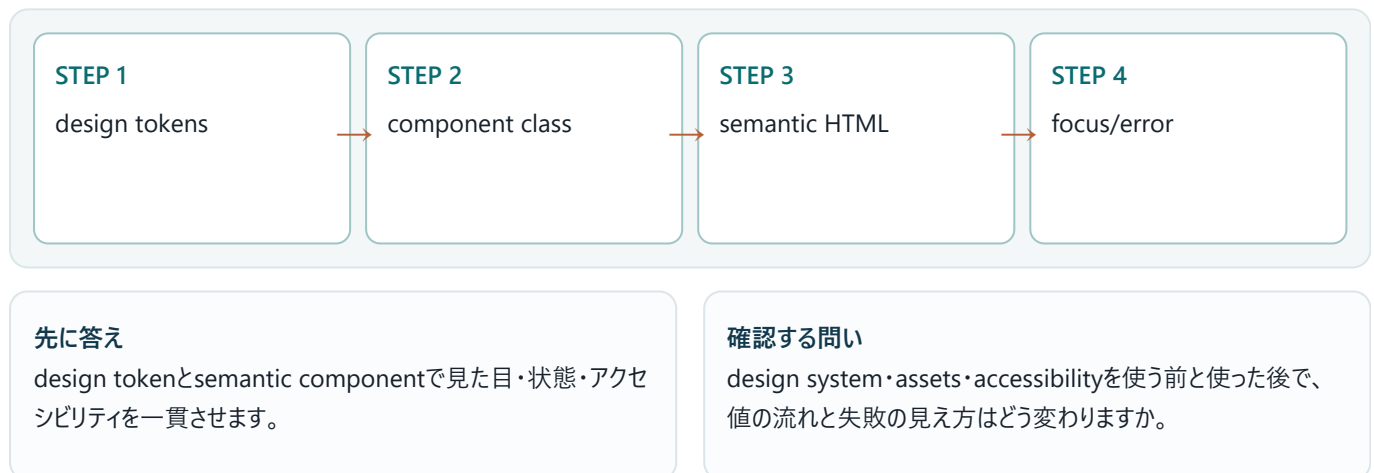
色、space、font、focus、button、form、card、errorを少数tokenとcomponentへまとめます。semantic HTML、label、error association、keyboard focus、contrastを最初から確認します。

この単元の到達点

- design tokens
- component class
- semantic HTML
- focus/error

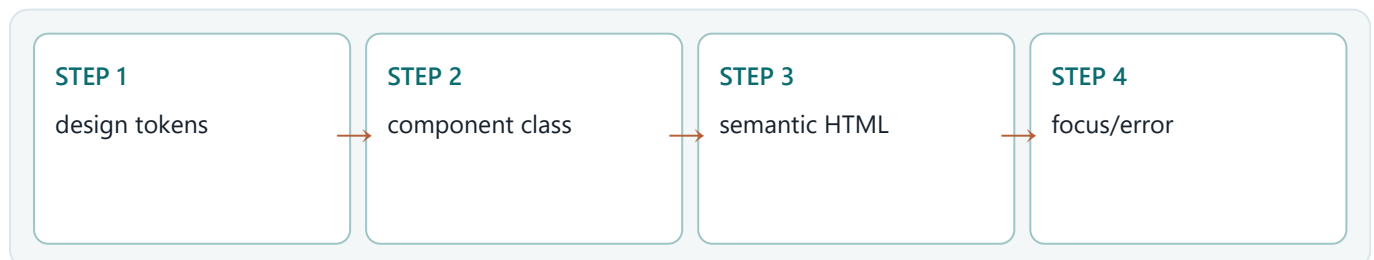
なぜバックエンドで必要か

CSSはasset!でbundleし、UI状態を文字・icon・色の複数手段で伝えます。delete buttonには具体的なaccessible nameを付けます。



UNIT 09・MENTAL MODEL

design system・assets・accessibilityを図でつかむ



色、space、font、focus、button、form、card、errorを少数tokenとcomponentへまとめます。semantic HTML、label、error association、keyboard focus、contrastを最初から確認します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	design tokens	design tokens — 入力と前提を確認し、境界を曖昧にしない。
2	component class	component class — 値がどの順で変換されるかを追跡する。
3	semantic HTML	semantic HTML — 失敗を再現し、型またはログから原因を特定する。
4	focus/error	focus/error — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

design system・assets・accessibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE

まず動く最小コード

design system・assets・accessibilityだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
:root{--ink:#102a3b;--accent:#0b7c7d;--danger:#a52a2a;--space-2:.5rem;--radius:.6rem}.field{display:grid;gap:.35rem}.field-error{color:var(--danger)}:focus-visible{outline:3px solid #f0a56a;outline-offset:2px}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>:root{--ink:#102a3b;--accent:#0b7c7d;--danger:#a52a2a;--space-2:.5rem;--ra</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>design system・assets・accessibility</code>	入力、処理、出力を分離して読みます。
<code>design system・assets・accessibility</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がdesign tokensの条件を満たすか確認する。
- ✓ component classを実行し、途中の型と状態を追う。
- ✓ semantic HTMLで失敗を成功経路から分離する。
- ✓ focus/errorにより、出力と副作用を検証する。

型を声に出す design system・assets・accessibilityに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

design system・assets・accessibility

focus ringが明確で、errorは項目直下とsummaryに表示され、色だけに依存しません。

観察ポイント

- design tokens
- component class
- semantic HTML

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
<label for="title">タイトル</label>
<input id="title" name="title" aria-describedby="title-help title-error" aria-invalid="true">
<p id="title-help">1～80文字</p><p id="title-error" role="alert">入力してください</p>
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

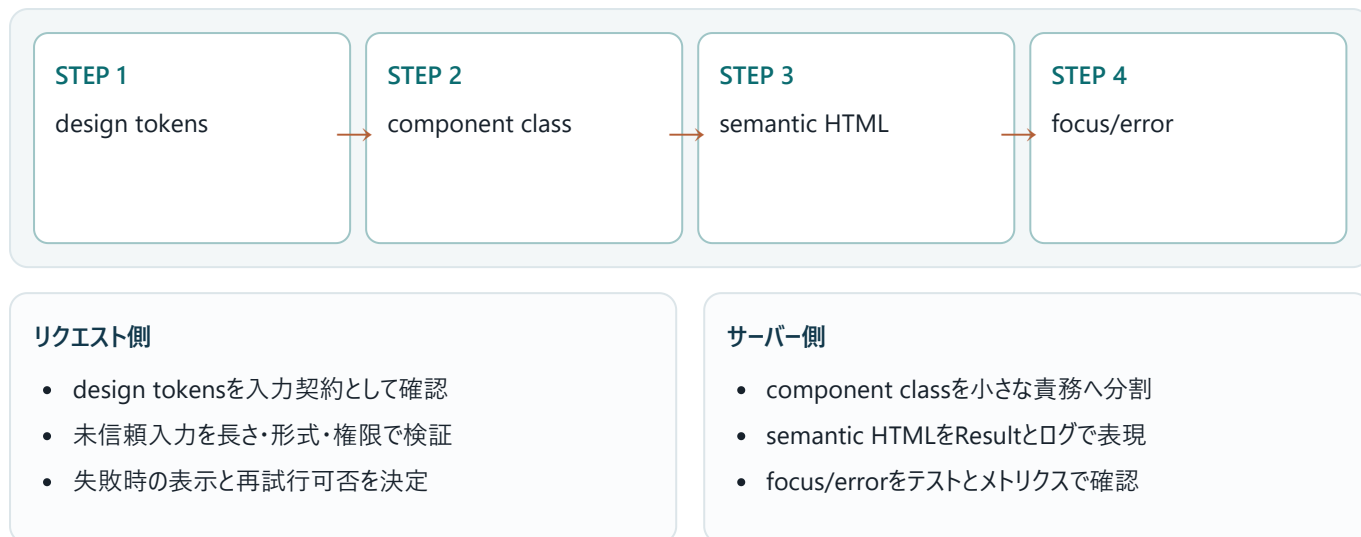
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09・BACKEND APPLICATION

バックエンドのどこで使うか

CSSはasset!でbundleし、UI状態を文字・icon・色の複数手段で伝えます。delete buttonには具体的なaccessible nameを付けます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	design tokensに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	component classの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	semantic HTMLまたはfocus/errorの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

design system・assets・accessibilityを再現して切り分ける

design system・assets・accessibilityの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: design tokens
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: component class
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: semantic HTML
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: focus/error

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	design tokens	design tokens — 入力と前提を確認し、境界を曖昧にしない。
2	component class	component class — 値がどの順で変換されるかを追跡する。
3	semantic HTML	semantic HTML — 失敗を再現し、型またはログから原因を特定する。
4	focus/error	focus/error — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 design tokenとsemantic componentで見た目・状態・アクセシビリティを一貫させます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

title inputへlabel、description、errorをaria-describedbyで関連付けてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
<label for="title">タイトル</label>
<input id="title" name="title" aria-describedby="title-help title-error" aria-invalid="true">
<p id="title-help">1～80文字</p><p id="title-error" role="alert">入力してください</p>
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ design tokenとsemantic componentで見た目・状態・アクセシビリティを一貫させます。
- ✓ design system・assets・accessibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

health・first run・診断

UNIT 10・CONCEPT

health・first run・診断

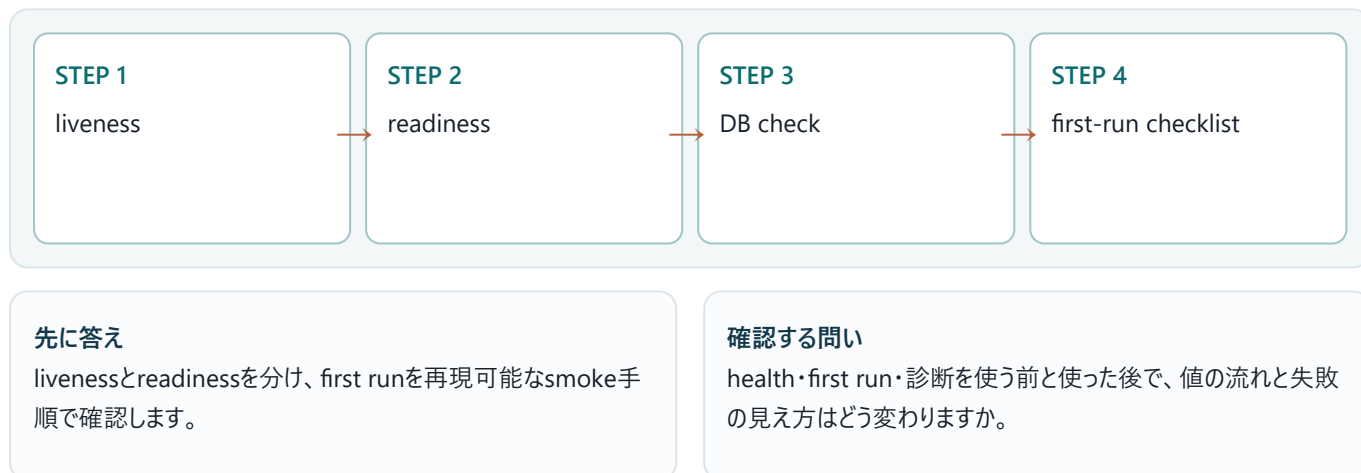
livenessはprocessが応答可能か、readinessは必要依存を使えるかを分けます。公開healthへ秘密や詳細errorを出さず、内部logで段階を追います。

この単元の到達点

- liveness
- readiness
- DB check
- first-run checklist

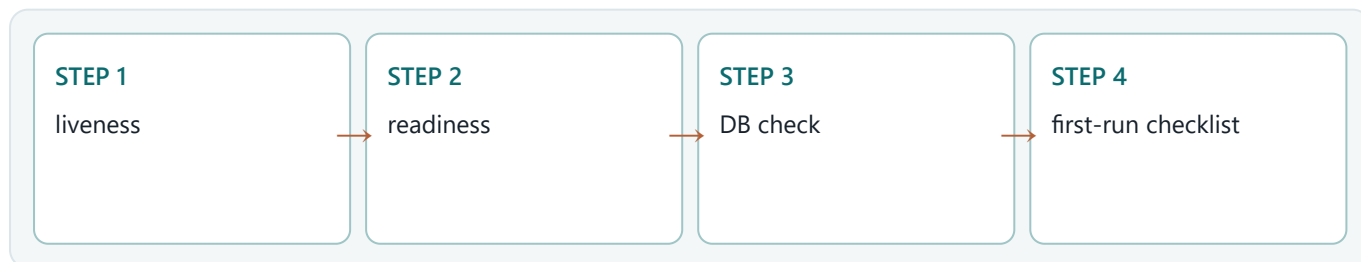
なぜバックエンドで必要か

DB障害でlivenessまで落とすと再起動loopを起こせます。Fly health checkの目的に合わせてendpointを選びます。



UNIT 10・MENTAL MODEL

health・first run・診断を図でつかむ



livenessはprocessが応答可能か、readinessは必要依存を使えるかを分けます。公開healthへ秘密や詳細errorを出さず、内部logで段階を追います。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	liveness	liveness — 入力と前提を確認し、境界を曖昧にしない。
2	readiness	readiness — 値がどの順で変換されるかを追跡する。
3	DB check	DB check — 失敗を再現し、型またはログから原因を特定する。
4	first-run checklist	first-run checklist — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

health・first run・診断とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

health・first run・診断だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
#[route(GET "/health")]async fn health()->Result<&'static str>{Ok("ok")}
#[route(GET "/ready")]async fn ready(cx:&Cx)->Result<&'static str>{db(cx).execute("SELECT 1",
()).await.map_err(|_|unavailable())?;Ok("ready")}
```

● ● ● 実行コマンド

```
topcoat dev
curl -i http://127.0.0.1:3000/health
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[route(GET "/health")]async fn health()->Result<&'static str>{Ok("ok")}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[route(GET "/ready")]async fn ready(cx:&Cx)->Result<&'static str>{db(cx).</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
health・first run・診断	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がlivenessの条件を満たすか確認する。
- ✓ readinessを実行し、途中の型と状態を追う。
- ✓ DB checkで失敗を成功経路から分離する。
- ✓ first-run checklistにより、出力と副作用を検証する。

型を声に出す health・first run・診断に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

GET /health → 200 ok
GET /ready → 200 ready または 503

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
cargo test
topcoat dev
curl -i /health
Browser: CSS loaded/register/login/create/edit/logout
Terminal: no panic/no secret
DB: expected rows only
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

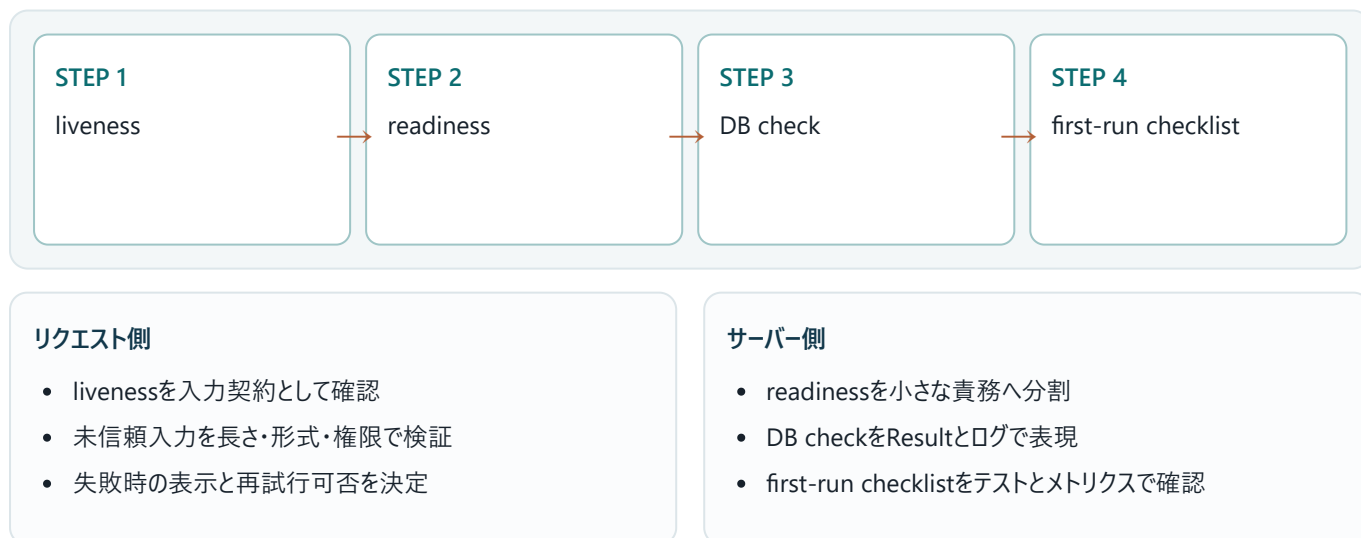
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

DB障害でlivenessまで落とすと再起動loopを起こせます。Fly health checkの目的に合わせてendpointを選びます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。

✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	livenessに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	readinessの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	DB checkまたはfirst-run checklistの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

health・first run・診断を再現して切り分ける

health・first run・診断の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: liveness
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: readiness
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: DB check
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: first-run checklist

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	liveness	liveness — 入力と前提を確認し、境界を曖昧にしない。
2	readiness	readiness — 値がどの順で変換されるかを追跡する。
3	DB check	DB check — 失敗を再現し、型またはログから原因を特定する。
4	first-run checklist	first-run checklist — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 livenessとreadinessを分け、first runを再現可能なsmoke手順で確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

起動、health、CSS、register、login、create、logoutをcurl/browserで確認するsmoke checklistを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
cargo test
topcoat dev
curl -i /health
Browser: CSS loaded/register/login/create/edit/logout
Terminal: no panic/no secret
DB: expected rows only
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ livenessとreadinessを分け、first runを再現可能なsmoke手順で確認します。
- ✓ health・first run・診断とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

RustNoteの設計書からhealthとlayoutまでを起動する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	要件・user story・受入条件	実装前に利用者の成果と拒否条件を観測可能な受入条件へします。
02	画面遷移・情報設計・wireframe	URLと画面状態を先に設計し、各ページの主目的と失敗表示を固定します。
03	domain model・schema・不変条件	同じ不変条件を型・境界検証・DB制約へ重ね、表現のずれをテストします。
04	project構造と依存方向	frameworkとDBを外側へ置き、domain規則を純粋Rustとして保ちます。
05	Cargo.toml・version固定・feature	依存は版・feature・責務・更新手順を一組で管理します。
06	Config・AppState・DB初期化	起動を設定→DB→migration→router→listenの段階に分け、途中失敗は安全に終了します。
07	migration・seed・test database	schema SQLは全環境で共通、seedは明示、test DBはケースごとに隔離します。
08	route inventoryと実装順	層を一度に全部作らず、利用者価値ごとのvertical sliceで完成させます。
09	design system・assets・accessibility	design tokenとsemantic componentで見た目・状態・アクセシビリティを一貫させます。
10	health・first run・診断	livenessとreadinessを分け、first runを再現可能なsmoke手順で確認します。

完了条件

- ✓ route表とER図がある
- ✓ Cargo.lockと版固定がある
- ✓ migrationがtestでも動く
- ✓ health・CSS・error表示を確認した

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. 要件・user story・受入条件を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. 画面遷移・情報設計・wireframeを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. domain model・schema・不変条件を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. project構造と依存方向を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. Cargo.toml・version固定・featureを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. Config・AppState・DB初期化を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. migration・seed・test databaseを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. route inventoryと実装順を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. design system・assets・accessibilityを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. health・first run・診断を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. 要件・user story・受入条件

要件・user story・受入条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 実装前に利用者の成果と拒否条件を観測可能な受入条件へします。

02. 画面遷移・情報設計・wireframe

画面遷移・情報設計・wireframeとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: URLと画面状態を先に設計し、各ページの主目的と失敗表示を固定します。

03. domain model・schema・不変条件

domain model・schema・不変条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 同じ不変条件を型・境界検証・DB制約へ重ね、表現のずれをテストします。

04. project構造と依存方向

project構造と依存方向とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: frameworkとDBを外側へ置き、domain規則を純粋Rustとして保ちます。

05. Cargo.toml・version固定・feature

Cargo.toml・version固定・featureとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 依存は版・feature・責務・更新手順を一組で管理します。

06. Config・AppState・DB初期化

Config・AppState・DB初期化とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 起動を設定→DB→migration→router→listenの段階に分け、途中失敗は安全に終了します。

07. migration・seed・test database

migration・seed・test databaseとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: schema SQLは全環境で共通、seedは明示、test DBはケースごとに隔離します。

08. route inventoryと実装順

route inventoryと実装順とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 層を一度に全部作らず、利用者価値ごとのvertical sliceで完成させます。

09. design system・assets・accessibility

design system・assets・accessibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: design tokenとsemantic componentで見た目・状態・アクセシビリティを一貫させます。

10. health・first run・診断

health・first run・診断とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: livenessとreadinessを分け、first runを再現可能なsmoke手順で確認します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
要件・user story・受入条件	実装前に利用者の成果と拒否条件を観測可能な受入条件へします。	受入条件をintegration test名へ変換し、route、SQL、画面の完成判断を一貫させます。
画面遷移・情報設計・wireframe	URLと画面状態を先に設計し、各ページの主目的と失敗表示を固定します。	Topcoat SSRではserverが完成HTMLを返します。validation error時は同じformへ値とerrorを再描画し、focusをerror summaryへ移します。
domain model・schema・不変条件	同じ不変条件を型・境界検証・DB制約へ重ね、表現のずれをテストします。	web DTOとDB rowをdomainから分け、password_hashとtoken_hashが画面用型へ入らないようにします。
project構造と依存方向	frameworkとDBを外側へ置き、domain規則を純粋Rustとして保ちます。	小規模なうちは一package内moduleにし、独立deployやcompile boundaryが必要になってからworkspace分割します。
Cargo.toml・version固定・feature	依存は版・feature・責務・更新手順を一組で管理します。	Cargo.lockをcommitし、Dependabot等の更新は一件ずつtest・smoke・migration互換を確認します。
Config・AppState・DB初期化	起動を設定 →DB→migration→router→listenの段階に分け、途中失敗は安全に終了します。	起動途中で失敗したらportを開かず終了し、どの段階で失敗したか構造化logへ残します。秘密値は表示しません。
migration・seed・test database	schema SQLは全環境で共通、seedは明示、test DBはケースごとに隔離します。	本番起動時にsample userを自動作成せず、開発seedをfeatureまたは別binaryへ分けます。
route inventoryと実装順	層を一度に全部作らず、利用者価値ごとのvertical sliceで完成させます。	route表はHTTP契約と実装順の両方を示し、各縦切りでweb・application・repository・view・testを同時に完成させます。
design system・assets・accessibility	design tokenとsemantic componentで見た目・状態・アクセシビリティを一貫させます。	CSSはasset!でbundleし、UI状態を文字・icon・色の複数手段で伝えます。delete buttonには具体的なaccessible nameを付けます。
health・first run・診断	livenessとreadinessを分け、first runを再現可能なsmoke手順で確認します。	DB障害でlivenessまで落とすと再起動loopを起こせます。Fly health checkの目的に合わせてendpointを選びます。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。